Informe de Auditoría de Sistemas

Examen de la Unidad I

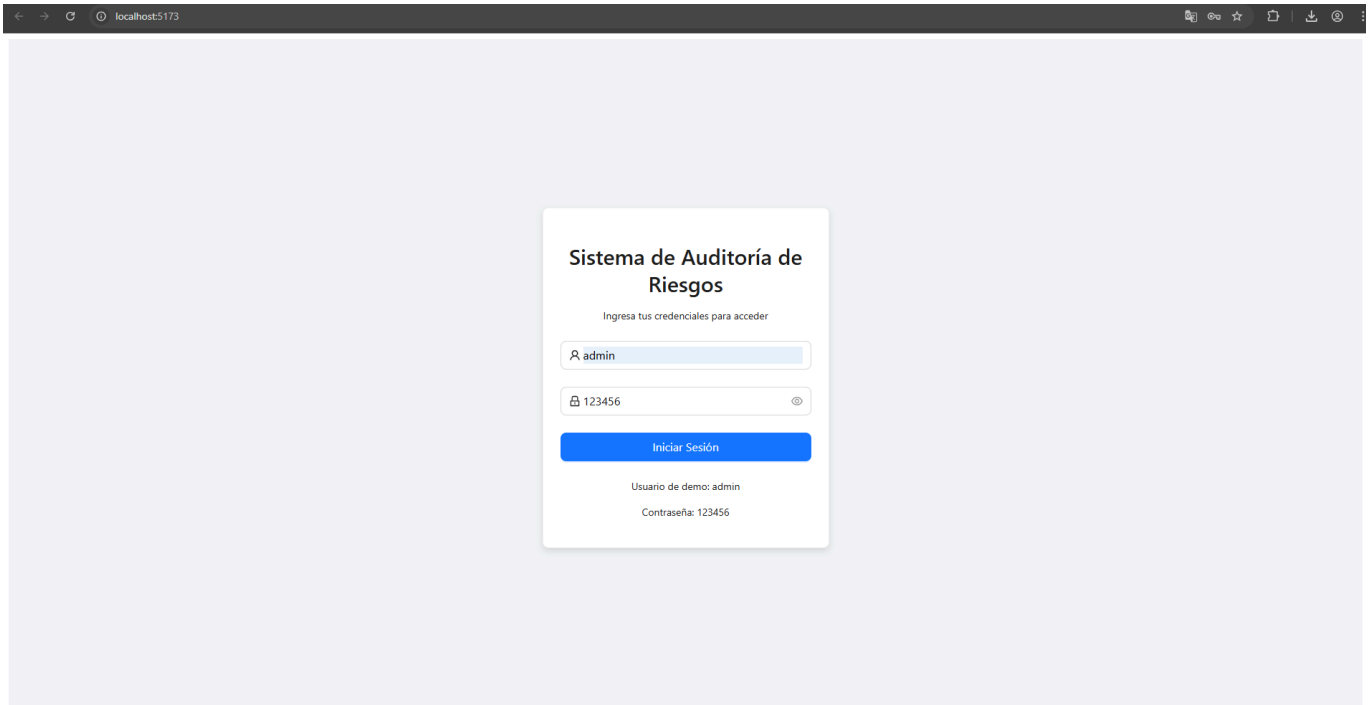
Nombres y apellidos: Piero Alexander Paja De la Cruz
Fecha: 22/04/26
URL GitHub: https://github.com/pieroalexanderppc/ExamenU1_Auditoria.git

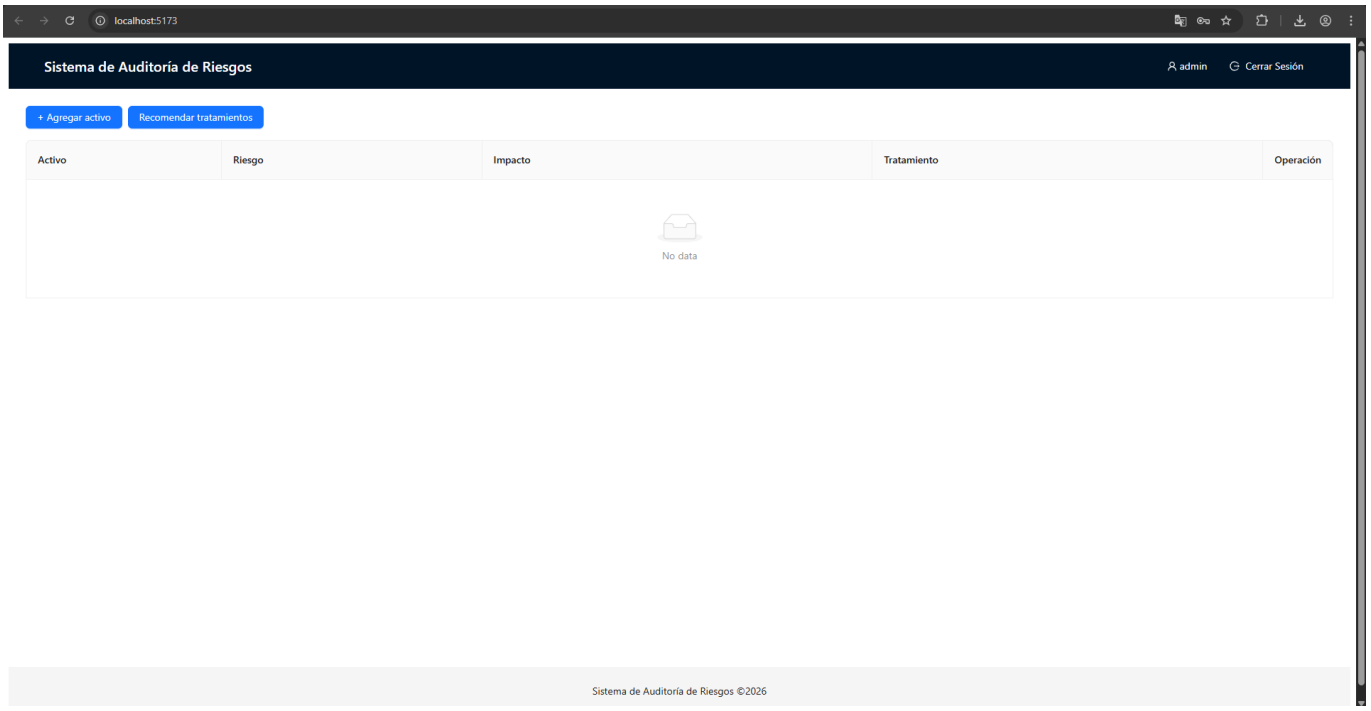
---

1. Proyecto de Auditoría de Riesgos

🔐 Login

Evidencia:





Descripción:
Se implementó un inicio de sesión ficticio sin base de datos mediante credenciales hardcodeadas (usuario: admin, contraseña: 123456).
La validación se realiza en el servicio `LoginService.js`, que genera un token simulado y lo almacena en `localStorage` para persistencia de sesión.
El componente `App.jsx` protege la vista principal y renderiza `Login.jsx` cuando no existe sesión activa.

🤖 Motor de Inteligencia Artificial

Evidencia:

app.py U ×
AuditoriaRiesgos > app.py > sugerir_tratamiento

```python
from openai import OpenAI
from flask import Flask, send_from_directory, request, jsonify, Response
import re


app = Flask(__name__)

# Ruta para servir el index.html desde la carpeta dist
@app.route('/',  methods=["GET",'POST'])
def serve_index():
    return send_from_directory('dist', 'index.html')

# Ruta para servir los archivos estáticos generados
@app.route('/<path:path>')
def serve_static(path):
    return send_from_directory('dist', path)

client = OpenAI(
    base_url = 'http://localhost:11434/v1',
    api_key='ollama', # required, but unused
)

@app.route('/analizar-riesgos', methods=['POST'])
def analizar_riesgos():
    data = request.get_json()  # Obtener datos JSON enviados al endpoint
    activo = data.get('activo')  # Extraer el valor del activo
    if not activo:
        return jsonify({"error": "El campo 'activo' es necesario"}), 400

    riesgos, impactos = obtener_riesgos(activo)  # Llamar a la función para obtener riesgos e impactos
    return jsonify({"activo": activo, "riesgos": riesgos, "impactos": impactos})
```

app.py U ×

AuditoriaRiesgos > app.py > sugerir_tratamiento

```python
@app.route('/sugerir-tratamiento', methods=['POST'])
def sugerir_tratamiento():
    data = request.get_json()  # Obtener datos JSON enviados al endpoint
    activo = data.get('activo')  # Extraer el valor del activo
    riesgo = data.get('riesgo')  # Extraer el valor del riesgo
    impacto = data.get('impacto')  # Extraer el valor del impacto

    # Verificar que todos los campos necesarios están presentes
    if not activo or not riesgo or not impacto:
        return jsonify({"error": "Los campos 'activo', 'riesgo' e 'impacto' son necesarios"}), 400

    # Combinar riesgo e impacto para formar la entrada completa para obtener_tratamiento
    entrada_tratamiento = f"{activo};{riesgo};{impacto}"
    tratamiento = obtener_tratamiento(entrada_tratamiento)

    return jsonify({"activo": activo, "riesgo": riesgo, "impacto": impacto, "tratamiento": tratamiento})


def obtener_tratamiento( riesgo ):
    response = client.chat.completions.create(
    model="ramiro:instruct",
    messages=[
    {"role": "system", "content": "Responde en español, eres una herramienta para gestion de riesgos de la iso 27000, el usuario, te
    {"role": "user", "content": "mi telefono movil;Acceso no autorizado;un atacante puede acceder a la información personal y confide
    {"role": "assistant",  "content": "Establecer un bloqueo de la pantalla de inicio que requiera autenticación con contraseña o hue
    {"role": "user", "content": riesgo }
    ]
    )
    answer = response.choices[0].message.content

```

app.py U ×

AuditoriaRiesgos > app.py > sugerir_tratamiento

```python
def obtener_tratamiento( riesgo ):
    return answer
def obtener_riesgos( activo ):
    response = client.chat.completions.create(
    model="ramiro:instruct",
    messages=[
    {"role": "system", "content": "Responde en español, eres una herramienta para gestion de riesgos de la iso 27000, el usuario, te
    {"role": "user", "content": "mi raspberry pi"},
    {"role": "assistant",  "content": """• **Acceso no autorizado**: terceros pueden acceder a la información almacenada o procesada

• **Pérdida o daño de datos**: los archivos y datos almacenados en el Raspberry Pi se pierden o dañan debido a un error en el sistema

• **Vulnerabilidades de seguridad**: El software o firmware instalados en el Raspberry Pi contienen vulnerabilidades de seguridad no

• **Inseguridad de la conexión**: la conexión del Raspberry Pi a la red local o internet no esté segura y un atacante intercepta dato

• **Fallos hardware**: daño debido a causas como sobrecalentamiento, sobrecarga eléctrica o errores en la manufactura, lo que lleva a
    {"role": "user", "content": activo }
  ]
)
    answer = response.choices[0].message.content
    patron = r'\*\*\s*(.+?)\*\*:\s*(.+?)\.(?=\s*\n|\s*$)'

    # Buscamos todos los patrones en la respuesta
    resultados = re.findall(patron, answer)

    # Separamos los resultados en dos listas: riesgos e impactos
    riesgos = [resultado[0] for resultado in resultados]
    impactos = [resultado[1] for resultado in resultados]

    return riesgos, impactos

#riesgos, impactos = obtener_riesgos("mi telefono movil")

if __name__ == '__main__':
    app.run(debug=True, host="0.0.0.0", port="5500")
```

Descripción:

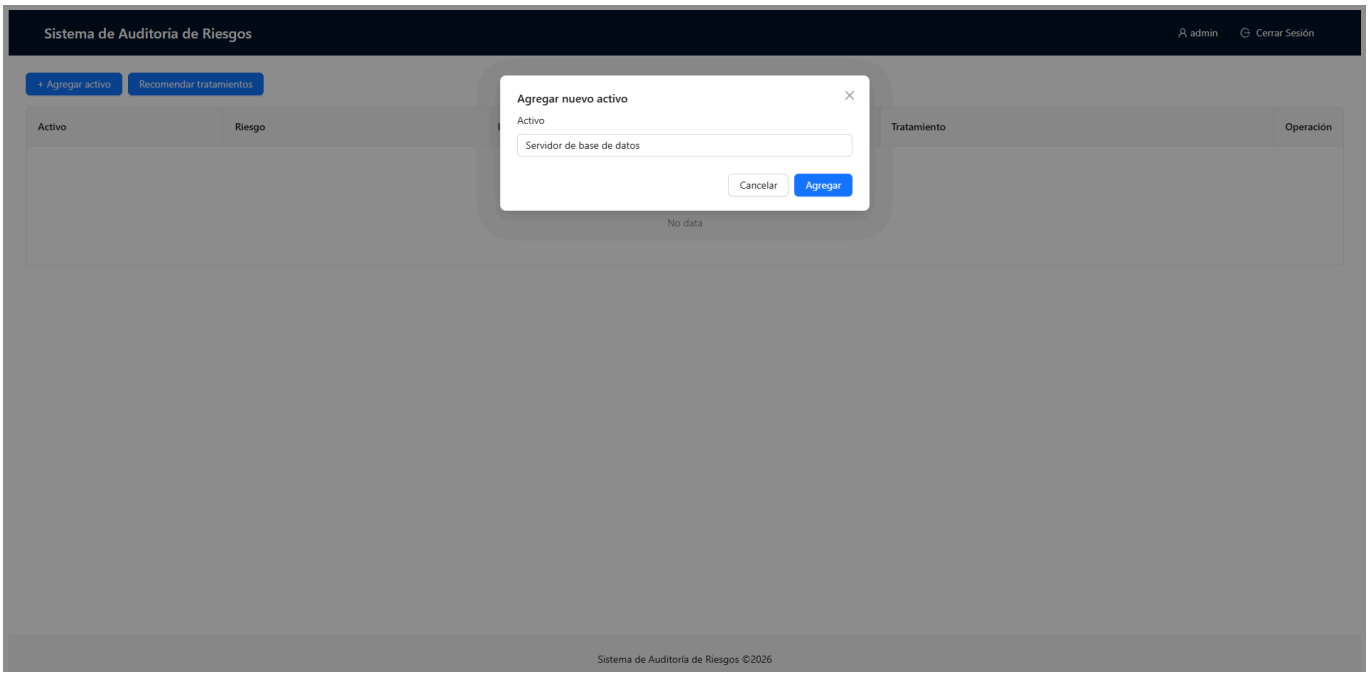
Se mejoró el motor de IA integrando un backend Flask que consume un modelo de lenguaje local por Ollama, usando cliente compatible OpenAI (`base_url=http://localhost:11434/v1`).

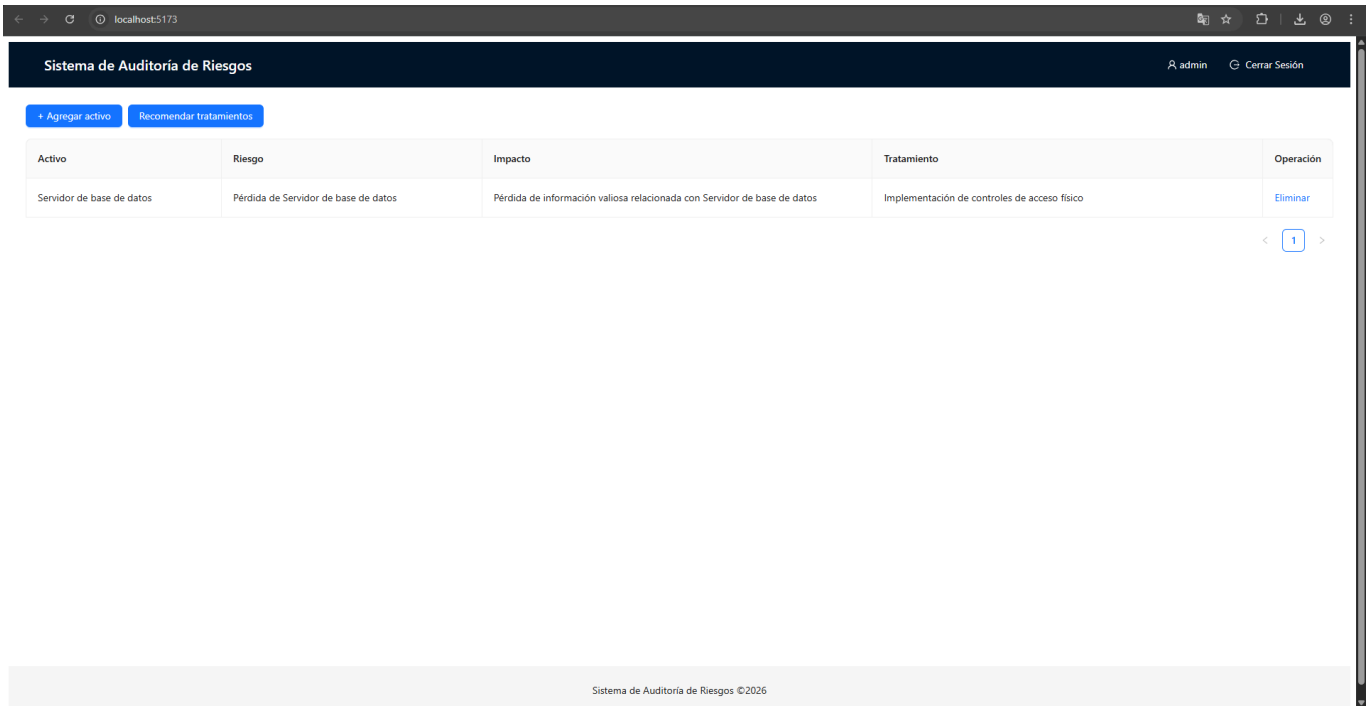
El endpoint `/analizar-riesgos` recibe un activo y devuelve lista de riesgos e impactos.

El endpoint `/sugerir-tratamiento` recibe activo, riesgo e impacto, y responde con una recomendación de tratamiento alineada a gestión de riesgos ISO 27001.

2. Hallazgos

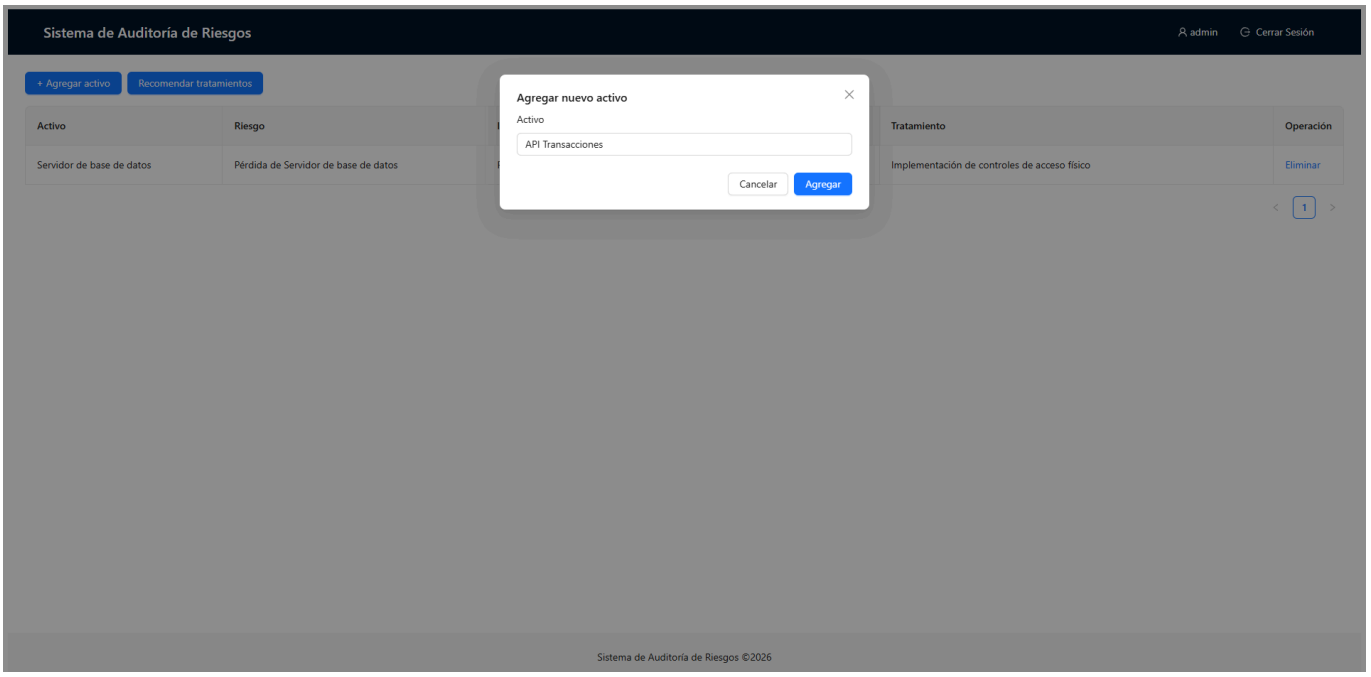
Activo 1: Servidor de base de datos

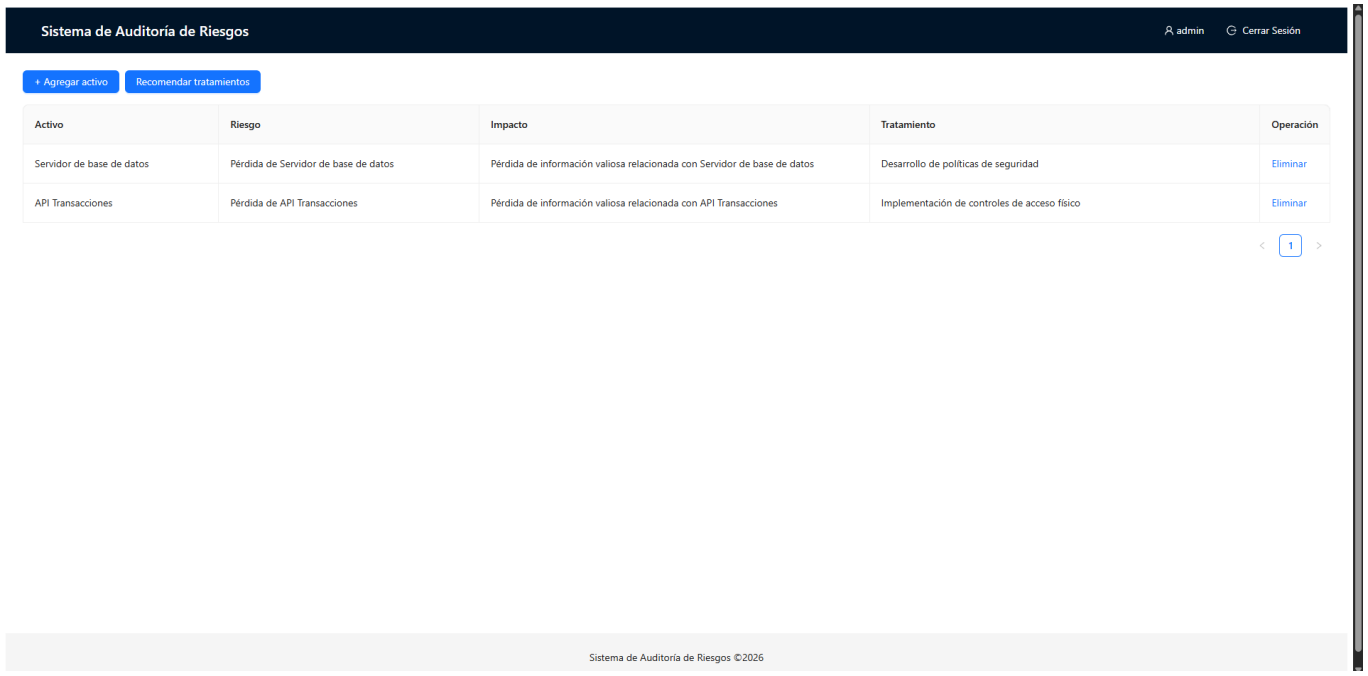




- **Evidencia:** Configuración sensible sin política explícita de rotación y endurecimiento; exposición potencial por credenciales y accesos no segmentados.
- **Condición:** Riesgo de acceso no autorizado y alteración de datos críticos de clientes/transacciones.
- **Recomendación:** Aplicar mínimo privilegio, segmentación de red, cifrado en reposo, rotación de credenciales y monitoreo continuo con alertas.
- **Riesgo:** Alta.

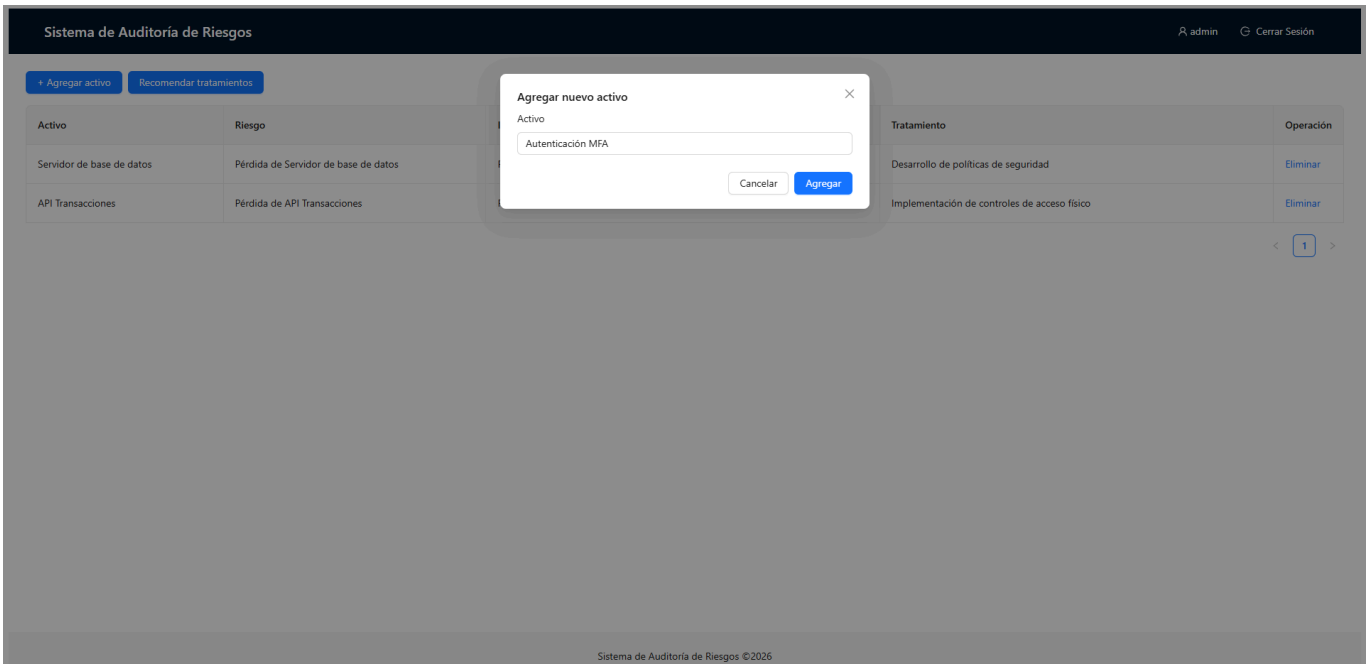
Activo 2: API Transacciones

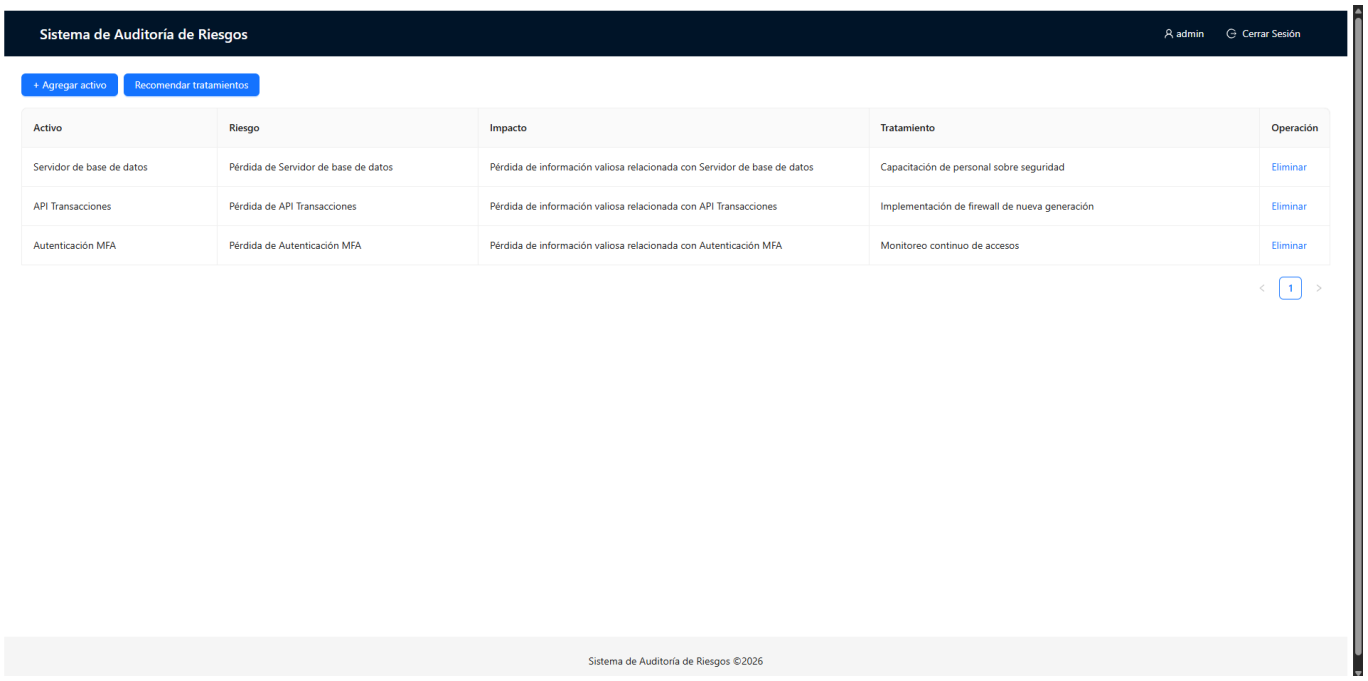




- **Evidencia:** Dependencia de tokens y servicios web para operaciones sensibles del negocio.
- **Condición:** Un control deficiente de autenticación/autorización puede permitir fraude, suplantación o abuso de endpoints.
- **Recomendación:** Implementar validación fuerte de tokens, rate limiting, registro de auditoría por operación y pruebas de seguridad API (OWASP API Top 10).
- **Riesgo:** Alta.

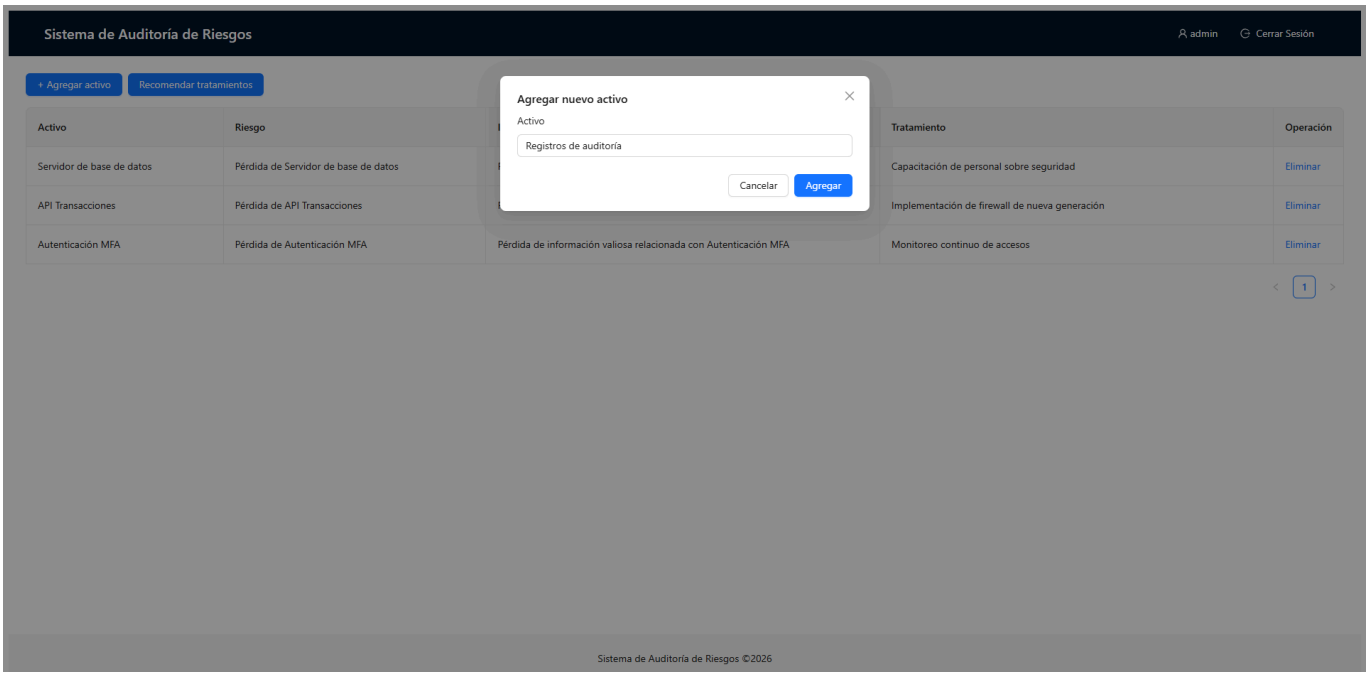
Activo 3: Autenticación MFA

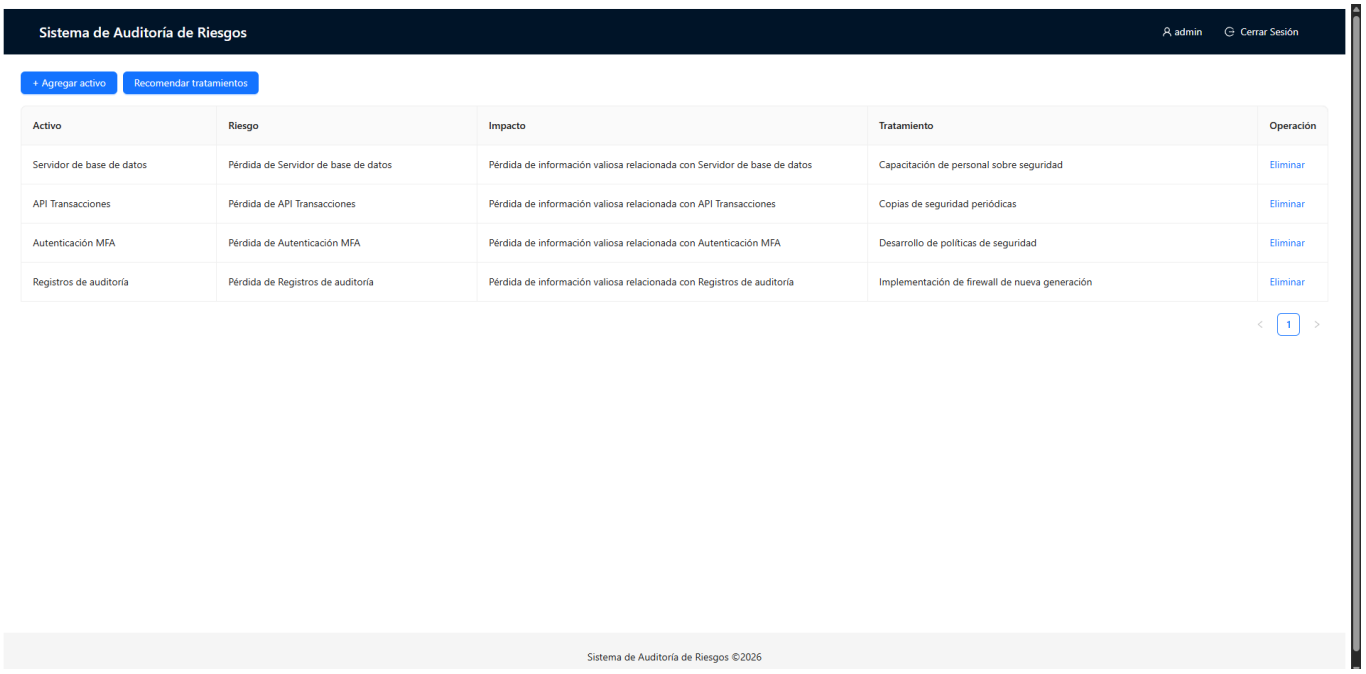




- **Evidencia:** La MFA está catalogada como activo crítico de seguridad en el entorno bancario.
- **Condición:** Si la MFA falla o está mal configurada, se incrementa el riesgo de secuestro de cuentas y acceso a información financiera.
- **Recomendación:** Exigir MFA para perfiles privilegiados y usuarios críticos, revisar políticas de enrolamiento y mecanismos antifraude para OTP/push.
- **Riesgo:** Media-Alta.

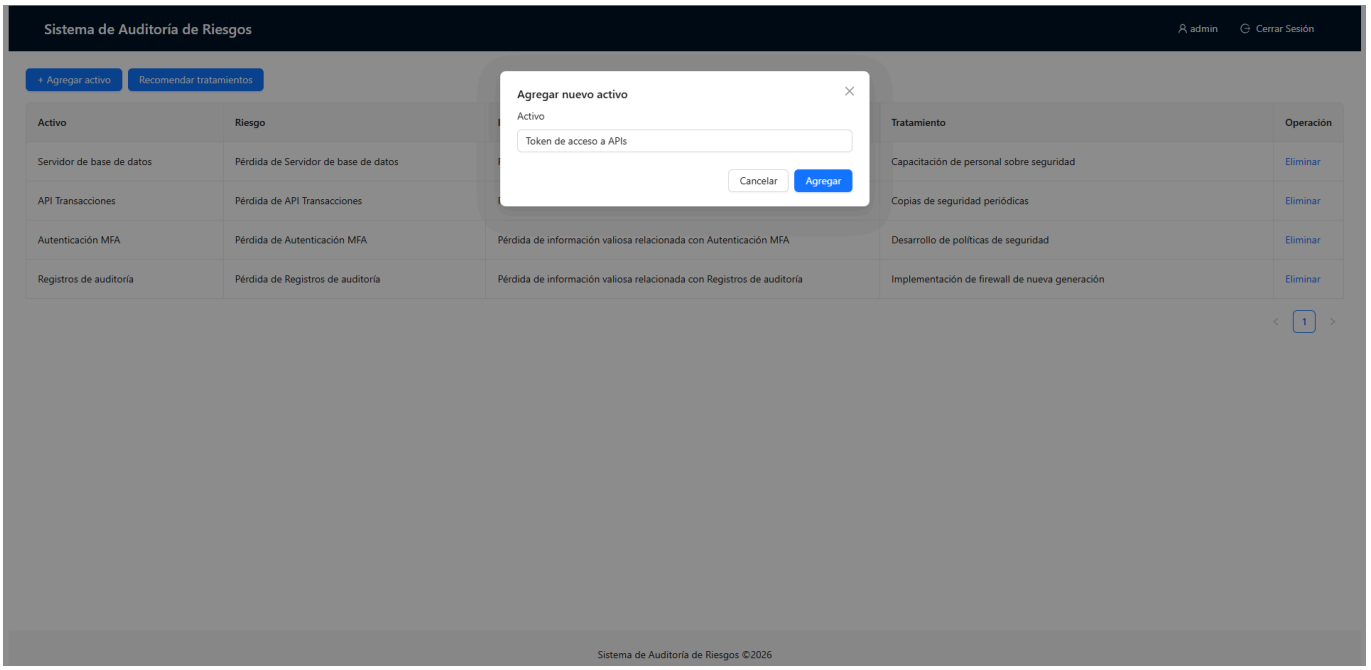
Activo 4: Registros de auditoría

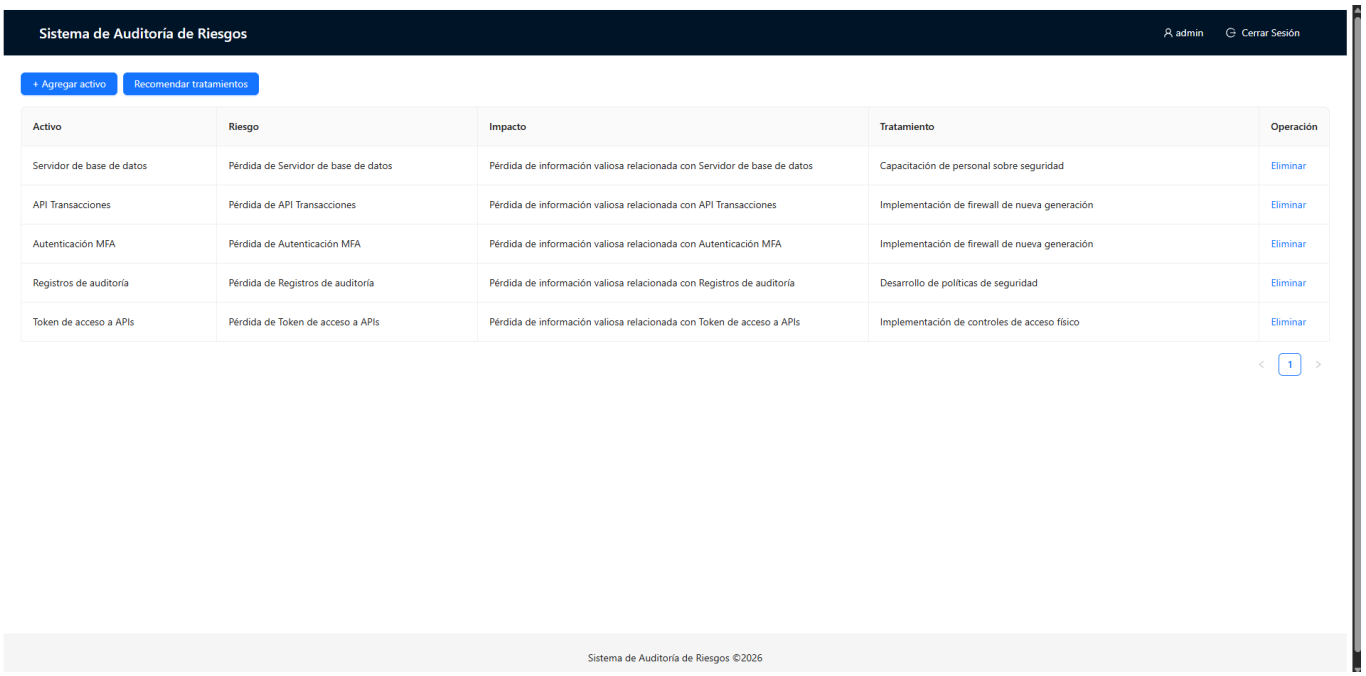




- **Evidencia:** Los logs son fuente principal para trazabilidad, detección de incidentes y cumplimiento normativo.
- **Condición:** Registros incompletos o manipulables impiden reconstruir incidentes y afectan investigación forense.
- **Recomendación:** Centralizar logs en SIEM, habilitar integridad/inmutabilidad, retención definida y correlación de eventos críticos.
- **Riesgo:** Alta.

Activo 5: Token de acceso a APIs





- **Evidencia:** Los tokens otorgan acceso directo a servicios financieros y procesos automatizados.
- **Condición:** Exposición de tokens por mala custodia o configuración puede derivar en compromiso total de APIs.
- **Recomendación:** Usar bóveda de secretos, expiración corta, rotación automática, scopes mínimos y revocación inmediata ante incidentes.
- **Riesgo:** Alta.

---

3. Conclusiones de Auditoría

1. Se implementó correctamente el inicio de sesión ficticio sin base de datos para fines académicos del examen.
2. Se incorporó un motor de IA local para análisis de riesgos y recomendación de tratamientos, cumpliendo el requerimiento de automatización.
3. Los cinco activos evaluados presentan riesgos relevantes para confidencialidad, integridad y disponibilidad; se prioriza remediación inmediata en API, base de datos, logs y gestión de tokens.

4. Las recomendaciones propuestas se alinean a controles de ISO 27001 (control de acceso, gestión de eventos, gestión de credenciales y monitoreo).